

Robust Agentic AI for Public Administration

A Multi-Agent Architecture and Controlled Fault Injection Evaluation

Radu-Mihai Gheorghe^{1,2} Petru-Liviu Bouruc² Ciprian Păduraru² Alin Ștefănescu^{2,3}

KES 2026 — 30th International Conference on Knowledge-Based and
Intelligent Information & Engineering Systems

¹UiPath, Romania ²University of Bucharest, Romania ³Institute for Logic and Data Science, Romania

1. Why fiscal services need coordination
2. The architecture
3. Fault injection at the boundaries
4. What the experiments show
5. Cost, and what it means for deployment

The question behind the paper

When realistic integration faults hit the service boundaries of an agentic system, how many reach the citizen unnoticed — and what decides that number?

The problem

Digital services exist, but they stay fragmented

Romania has digitised its fiscal services. Each portal keeps its own interface, vocabulary and procedural logic.

THREE PORTALS

- **SPV** — declarations, fiscal record
- **Ghişeu.ro** — payments
- **RO e-Factura** — national e-invoicing

ONE FREELANCER, ONE YEAR

- declare annual income (D212)
- pay pension and health contributions
- register a rental contract
- issue an electronic invoice

Four procedures, three portals, no shared state.

The hard part is coordination and interpretation of the rules, not arithmetic.

Two objectives, two audiences

FOR THE TAXPAYER

One conversational entry point that hides the split between SPV, Ghișeul.ro and RO e-Factura, so routine tasks no longer mean moving between portals.

FOR ANAF, WHO WOULD HOST IT

Behaviour that stays coherent when a dependency is slow, partially available, or returns malformed data. Incoherent answers in a regulated service cost credibility.

The architecture addresses the first objective. The fault injection study addresses the second, and takes up most of this talk.

1. A **multi-agent architecture** for public administration: an Entry Agent classifies intent and routes to one of five domain agents, over four shared services. Instantiated for Romanian tax administration on the UiPath Python SDK and LangGraph.
2. A **fault injection framework** over 19 external service boundary operations, with four chaos operators — timeout, delay, partial response, corruption — applied at configurable rates and replayable from a seed.
3. The measured result: **about 63% of injected faults propagate silently**, a figure that follows from the operator mix and the shape of the response schema rather than from an implementation bug. Code and evaluation artefacts are public.

AGENTIC ORCHESTRATION

LangGraph models control flow as a typed state machine: explicit and auditable, unlike open-ended conversation. That matters in a regulated service.

RETRIEVAL (RAG)

Tax law is specific and amended often. Retrieval grounds explanations in the Romanian Tax Code and stays separate from the deterministic computation.

CHAOS ENGINEERING

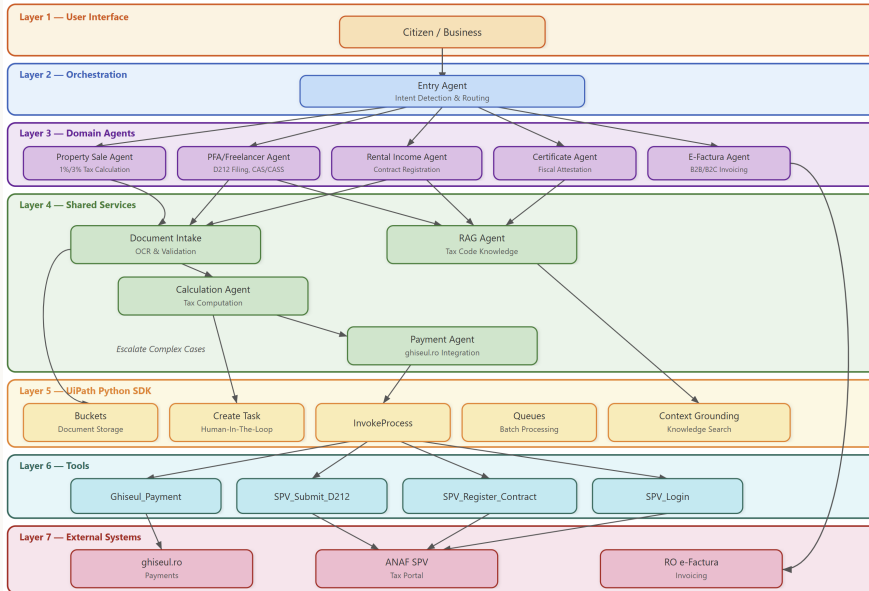
Fault injection has reached LLM agents recently, but in generic or synthetic testbeds rather than in an operational public service.

The gap

No prior work combines an operational multi-agent architecture for a national fiscal setting with a boundary-level fault study that explains *why* faults pass silently, in terms of response schemas and branching predicates.

The architecture

Seven layers, one state machine



Orchestration: one entry, confidence-gated routing



- The LLM classifies; it does not decide the route. A fixed table turns the predicted intent into a graph node, so the routing step itself stays inspectable.
- Below the confidence threshold the request goes to clarification, even when an intent was assigned. Dispatching the wrong specialist here means computing the wrong tax.

Five workflows over four shared services

DOMAIN AGENTS

- **PFA / Freelancer** — *Declarația Unică* (D212); pension contribution 25% above 12 minimum salaries (48,600 RON in 2026), health 10% above 6
- **Property sale** — 3% under three years of ownership, 1% at or above
- **Rental income** — contract registration with ANAF, 10% flat tax
- **Certificate** — *certificat de atestare fiscală*
- **e-Factura** — B2B and B2C invoicing, XML generation, status monitoring

SHARED SERVICES

- **Document intake** — OCR and schema validation; below 80% confidence the document goes to manual review
- **RAG** — retrieval over indexed sections of the Romanian Tax Code, so an explanation can be traced to a source
- **Calculation** — decimal arithmetic, no floating point; rates read from configuration and updated yearly without code changes
- **Payment** — builds the request, returns a Ghișeul.ro redirect URL

Design rule: the model interprets, deterministic code computes.

Why this substrate

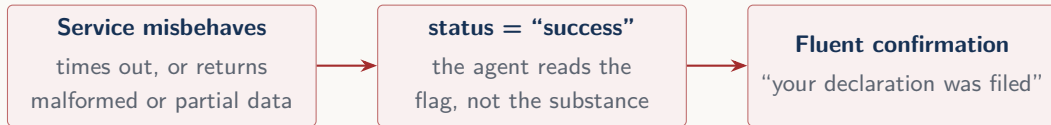
A bare orchestration library leaves model access, retrieval, retry logic, escalation and document storage to be assembled separately. The UiPath Python SDK supplies them next to LangGraph, in one execution environment.

- UiPathChat — model access through the platform gateway (GPT-4o)
- ContextGroundingRetriever — retrieval over indexed knowledge bases
- Buckets, queues and human review tasks for storage and escalation
- Government boundaries are **mocked**, but keep the request and response schemas of the real services
- In production these become robotic processes; some portals still require browser interaction rather than an API

The mocks are what make a controlled robustness study possible without live access to government systems.

Fault injection at the boundaries

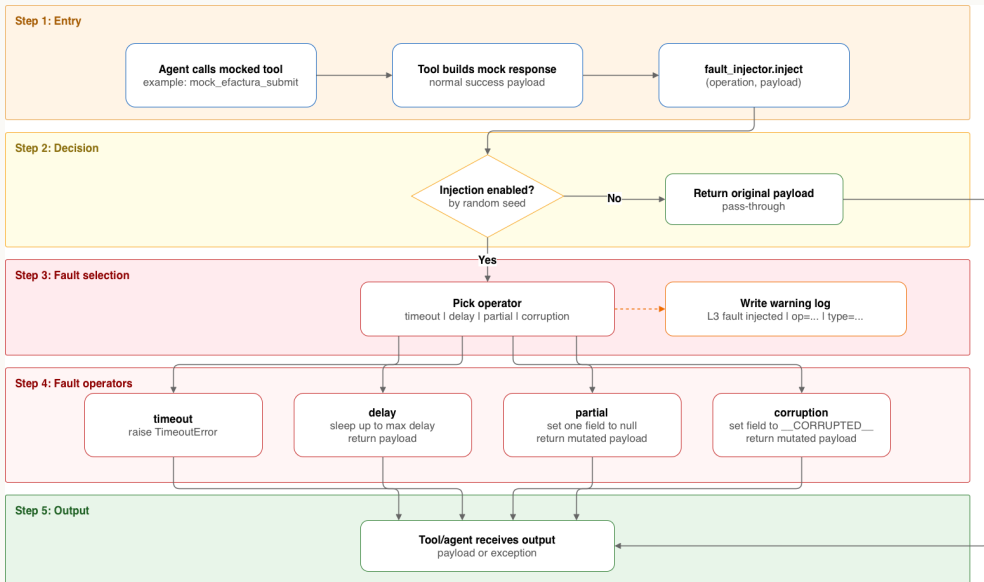
The failure mode that matters here



An agent that trusts a service-reported success flag and looks no further will build a confident answer on top of a broken response. The conversation reads as normal while invalid state passes underneath it.

In a regulated setting this is worse than a visible crash: nothing signals that anything went wrong.

The injector: one draw per boundary crossing



Four chaos operators

Operator	Effect on the call	Payload	Visible to the agent?
Timeout	raises <code>TimeoutError</code>	none returned	always
Delay	sleeps $\mathcal{U}[1, \Delta t_{\max}]$ ms	unchanged	never
Partial	one random field set to <code>None</code>	mutated	usually not
Corruption	one random field set to a sentinel	mutated	usually not

- Three parameters: seed s for deterministic replay, per-call injection rate η , delay bound $\Delta t_{\max}$. At $\eta = 0$ the nominal system is recovered unchanged.
- 19 instrumented boundary operations: 8 function-level mock tools and 11 class-method wrappers across three mocked external systems.
- Operators are drawn uniformly, so each accounts for a quarter of injected faults. The injector is a separate wrapper layer; the nominal service logic is untouched.

Why the faults are silent

Agent control flow usually turns on a single field: a predicate such as `status == "success"`. The rest of the response object carries content the branch never inspects.

`status`
branching field

`message`
content only

`id`
content only

`timestamp`
content only

A single-field perturbation that misses `status` leaves the predicate true, so the agent takes the success branch anyway. For a response with $|\mathcal{F}|$ fields, one of which gates control flow:

$$P(\text{silent}) = 1 - \frac{1}{|\mathcal{F}|}$$

Expected silent rate

$$\underbrace{0.25}_{\text{delay}} + \underbrace{2 \times 0.25 \times 0.75}_{\text{partial, corruption}} = \mathbf{0.625}$$

Timeout, the remaining quarter, always raises.

Results

PROTOTYPE

- Python 3.11, LangGraph 0.2, GPT-4o via the UiPath LLM Gateway
- No live network traffic: every government call goes through the mocked boundary layer

RUNS

- $N = 100$ boundary calls, each an independent trial with at most one injected fault
- Sampled across the 8 mock tools, weighted by call frequency in the five demo workflows
- $\eta \in \{0, 0.25, 0.50, 0.75, 1.00\}$, $s = 17$, $\Delta t_{\max} = 250 \text{ ms}$

CLASSIFYING AN OUTCOME

Detectable

The response signals failure: an exception propagates, or an explicit error or degraded-service notice reaches the user.

Silent pass-through

Anything else. The citizen-facing answer looks normal.

The unit of analysis is the call, not the session — a full workflow crosses several boundaries.

One trace, in full

A citizen reports PFA income of 150,000 RON and asks what is owed.

1. Entry Agent classifies pfa_d212_filing, confidence 0.95
2. PFA Agent computes pension 9,900 RON and health 1,980 RON, total 11,880 RON
3. On the confirmation turn, the SPV submission call is intercepted at $\eta = 1.0$, $s = 17$

```
[WARN] fault injected | op=mock_spv_submit_d212  
| type=partial
```

The message field of SPVSubmissionResult was set to None. status stayed "success", so the agent followed the success branch and answered:

"The D212 declaration was successfully submitted! ID: D212-01B66C85C3, Timestamp: 2026-03-27T22:20:32."

Four fields, one of them gating. A single-field fault stays invisible with probability 0.75. Nothing about this trace is accidental.

About 63%, at every injection rate

η	Faults	TO	DL	PT	CR	Silent	Detectable
0.00	0	0	0	0	0	0 (—)	0 (—)
0.25	25	6	6	7	6	16 (64%)	9 (36%)
0.50	50	12	13	12	13	32 (64%)	18 (36%)
0.75	75	19	19	18	19	47 (63%)	28 (37%)
1.00	100	25	25	25	25	63 (63%)	37 (37%)

$N = 100$ calls, $s = 17$, $\Delta t_{\max} = 250$ ms. TO, DL, PT, CR: timeout, delay, partial, corruption.

- The measured rate sits on the predicted 0.625. Raising η raises the *count* of silent faults, not their proportion.
- Delay faults added 125.5 ± 72.2 ms per affected call, matching the mean of $\mathcal{U}[1, 250]$. At $\eta = 0.50$ that is roughly 65 ms per scenario.
- Because the rate follows from the operator mix and the schema, it can be predicted before the system runs — and lowered by changing what the branch inspects.

Cost is not the obstacle

PER SESSION

\$0.05

one classification, two to three
domain turns, one retrieval step

SMALL OFFICE

\$51

per month, at 50 sessions a day

LARGE AGENCY

\$5,060

per month, at 5,000 sessions a
day

- About 20,000 input and 2,100 output tokens per session, priced at \$1.25 per million input and \$10.00 per million output tokens, over 22 working days a month. Licensing, infrastructure and maintenance are excluded.
- Cost scales linearly and stays modest. Routing with a smaller model, keeping the capable one for domain reasoning, pushes it lower.

What stands in the way of deployment is assurance, not arithmetic.

Conclusion

Interface coherence is not evidence of correctness

WHAT WE FOUND

- An operational multi-agent architecture covering five Romanian fiscal workflows, on LangGraph and the UiPath Python SDK
- A silent pass-through rate near 63%, stable across injection rates and matching a closed-form prediction
- The cause sits in response schemas and branching predicates, not in incidental bugs
- Inference cost is modest at institutional volumes

A fault that never changes the branch will never announce itself. Validation has to happen at the output, where the contract is.

WHAT COMES NEXT

- **Output contracts** — check responses against explicit validity conditions before they reach the citizen: missing submission identifiers, invalid payment codes, incomplete certificate responses
- **Trace-based assurance** — structured message-action traces, deterministic replay, action-level governance
- **Beyond taxation** — other public-service domains reusing the orchestration and shared services layers

Thank you

Radu-Mihai Gheorghe

`github.com/radugheo/agentic-public-administration`